Master Topic and Question List From All Sessions

Deepak, I reviewed the sessions, transcripts, follow-up questions, and preparation discussions you shared from the beginning up to the latest session. Below is a consolidated syllabus arranged as **main topic → subtopics → questions to prepare**.

I have separated the **Python backend training syllabus** from the additional topics you studied, such as VBA, AI, and HCI.

Part A: Python Backend Development Sessions

1. Backend and Client-Server Fundamentals

Subtopics

- Client and server
- Frontend and backend
- Request-response model
- Stateless communication
- URL and API endpoints
- Browser, API server, and database communication
- How a request reaches a server
- What happens inside the server
- Application runtime
- Application dependencies
- Environment and configuration
- Ports and processes
- Development, testing, and production environments

Questions

1. What is a client?
2. What is a server?
3. What is a backend service?
4. What is the difference between frontend and backend?
5. How does a browser communicate with a server?
6. What happens when a user enters a URL in the browser?
7. How does an HTTP request reach the application?
8. What happens inside the server after receiving a request?
9. What is an API?

10. What is an API endpoint?
11. What is stateless communication?
12. What is an application runtime?
13. What are runtime dependencies?
14. What is the difference between environment and configuration?
15. What is the purpose of a port?
16. Can two applications run on the same port?
17. What is the difference between development, testing, staging, and production?
18. How does an application run in a production environment?

This was one of the earliest session themes, especially the movement of a request from the user through the backend application and its internal processing layers. [Past Chat...gate the m | Chat], [Past Chat...uring APIs | Chat]

---

2. HTTP Request and Response Lifecycle

Subtopics

- HTTP and HTTPS
- HTTP methods
- Request URL
- Request headers
- Request body
- Query parameters
- Path parameters
- Cookies
- Status codes
- Content types
- HTTP lifecycle
- Routing
- Request processing chain
- Browser Developer Tools

HTTP methods

- GET
- POST
- PUT
- PATCH
- DELETE

Important status codes

- 200 OK
- 201 Created
- 202 Accepted
- 204 No Content
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 409 Conflict
- 422 Unprocessable Entity
- 429 Too Many Requests
- 500 Internal Server Error
- 503 Service Unavailable

Questions

1. What is HTTP?
2. What is the difference between HTTP and HTTPS?
3. What happens during an HTTP request?
4. What is the difference between request and response?
5. What is the difference between headers and body?
6. What information is stored in HTTP headers?
7. What is a request payload?
8. What is the difference between path parameters and query parameters?
9. What is routing?
10. How does a router identify the correct endpoint?
11. What is the difference between GET and POST?
12. What is the difference between PUT and PATCH?
13. When should DELETE return 204?
14. What is the difference between 401 and 403?
15. What is the difference between 400 and 422?
16. When should an API return 500?
17. What can be examined in the browser Network tab?
18. How do you trace a request using browser Developer Tools?

The training roadmap explicitly included HTTP lifecycle, REST methods, routing, headers, body, status codes, validation, and practical use of browser Developer Tools. [Past Chat...uring APIs | Chat]

---

3. REST API Design

Subtopics

- REST architecture
- Resource-oriented URLs
- CRUD operations
- HTTP methods
- Idempotency
- Request contracts
- Response contracts
- API versioning
- Pagination
- Filtering
- Sorting
- Error responses
- API documentation

Questions

1. What is REST?
2. What is a REST API?
3. What is a resource in REST?
4. How should REST endpoints be named?
5. What are CRUD operations?
6. How are CRUD operations associated with HTTP methods?
7. What is idempotency?
8. Which HTTP methods are idempotent?
9. Why is GET expected to be idempotent?
10. Is POST idempotent?
11. How can POST be made idempotent?
12. What is an idempotency key?
13. What is API versioning?
14. Why do APIs need pagination?
15. What is the difference between offset and cursor pagination?
16. How should an API return validation errors?
17. What should a consistent API error response contain?
18. What is the difference between synchronous and asynchronous API responses?

---

4. FastAPI

Subtopics

- What FastAPI is
- ASGI
- Uvicorn
- Starlette
- Pydantic
- Application creation
- Route decorators
- Path operations
- Path parameters
- Query parameters
- Request bodies
- Response models
- Automatic validation
- Swagger UI
- ReDoc
- Dependency injection
- Middleware
- Exception handling
- Background tasks
- Async endpoints
- Database integration
- Application structure

Basic architecture

```
Client
  ↓
FastAPI Route
  ↓
Service Layer
  ↓
Repository/Data Layer
  ↓
Database
```

Questions

1. What is FastAPI?
2. Why is FastAPI used?

3. What is ASGI?
4. What is Uvicorn?
5. What is Starlette?
6. What is Pydantic?
7. How do you create a FastAPI application?
8. How do you run FastAPI using Uvicorn?
9. What does main:app mean?
10. What does --reload do?
11. How do you create a GET endpoint?
12. How do you create a POST endpoint?
13. How do path parameters work?
14. How do query parameters work?
15. How does FastAPI validate a request body?
16. What is a response model?
17. What is dependency injection?
18. What is Depends()?
19. How does middleware work?
20. How do you implement global exception handling?
21. How do you connect FastAPI to a database?
22. How do you manage database sessions?
23. What is the difference between def and async def endpoints?
24. When should background tasks be used?
25. How does FastAPI generate Swagger documentation?
26. How should a production FastAPI application be structured?
27. What is the difference between FastAPI, Flask, and Django?
28. Why is FastAPI considered fast?

FastAPI preparation expanded into routes, parameters, Pydantic validation, application layers, database integration, middleware, and async processing. [Past Chat...al-text="t | Chat], [Past Chat...ero-Shot P | Chat]

---

5. Request and Response Validation

Subtopics

- Input validation
- Output validation
- Required and optional fields
- Type validation
- Format validation
- Business validation

- Pydantic models
- Custom validators
- Error handling
- Validation error responses
- Sensitive-data filtering

Questions

1. Why is request validation required?
2. What is response validation?
3. What is the difference between schema validation and business validation?
4. How does Pydantic perform type validation?
5. What happens when invalid data is sent to FastAPI?
6. How do you define required and optional fields?
7. How do you validate email addresses?
8. How do you validate minimum and maximum values?
9. How do you create custom validators?
10. Why should API responses also be validated?
11. How do you prevent sensitive fields from appearing in responses?
12. What is the correct status code for invalid input?
13. Where should business validation be implemented?
14. How should validation errors be logged?

Request and response validation was identified as a completed and important training area, particularly in relation to authentication APIs and FastAPI. [Past Chat...ashraj Pat | Chat], [Past Chat...gate the m | Chat]

Part B: Python Programming

6. Python Fundamentals

Subtopics

- Variables
- Data types
- Operators
- Conditional statements
- Loops
- Functions
- Arguments
- Return values

- Lists, tuples, sets, and dictionaries
- String operations
- Exception handling
- File handling
- Modules and packages
- Imports
- Classes and objects

Questions

1. What is Python?
2. Why is Python used in backend development?
3. Is Python compiled or interpreted?
4. What are mutable and immutable objects?
5. What is the difference between list and tuple?
6. What is the difference between set and dictionary?
7. What are positional and keyword arguments?
8. What are *args and **kwargs?
9. What is a lambda function?
10. What is exception handling?
11. What is the purpose of try, except, else, and finally?
12. What is the difference between a module and a package?
13. What is the purpose of __init__.py?
14. What does if __name__ == "__main__" mean?
15. How does Python manage memory?

---

7. Object-Oriented Programming

Subtopics

- Classes
- Objects
- Attributes
- Methods
- Constructors
- Instance methods
- Class methods
- Static methods
- Encapsulation
- Inheritance
- Polymorphism

- Abstraction
- Composition
- Abstract base classes
- Interfaces in Python

Questions

1. What is OOP?
2. What is a class?
3. What is an object?
4. What is the difference between class and object?
5. What is self?
6. What is __init__?
7. What is encapsulation?
8. What is inheritance?
9. What is polymorphism?
10. What is abstraction?
11. What is method overriding?
12. What is the difference between inheritance and composition?
13. What is an abstract class?
14. What is the difference between instance, static, and class methods?
15. When should OOP be used in backend applications?
16. What is the difference between OOP and functional programming?

Python basics and OOP were repeatedly referenced as foundational prerequisites in the sessions. [Past Chat...ashraj Pat | Chat], [Past Chat...tes 8 seco | Chat], [Past Chat...ith multip | Chat]

---

8. Functional Programming

Subtopics

- Pure functions
- Immutability
- First-class functions
- Higher-order functions
- Lambda functions
- map
- filter
- reduce
- Side effects
- Closures

- Decorators

Questions

1. What is functional programming?
2. What is a pure function?
3. What is a side effect?
4. What is immutability?
5. What is a higher-order function?
6. What is a closure?
7. What is a decorator?
8. What is the difference between OOP and functional programming?
9. When should functional programming be used?
10. Can OOP and functional programming be used together?

---

9. Python Environment and Package Management

Subtopics

- Python interpreter
- Python versions
- Virtual environments
- venv
- pip
- uv
- PyPI
- requirements.txt
- pyproject.toml
- Dependency installation
- Dependency locking
- Version constraints
- Environment activation
- Reproducible builds

Questions

1. What is a virtual environment?
2. Why is a virtual environment required?
3. How do you create and activate a venv?
4. What is pip?
5. What is PyPI?
6. How does pip install work?

7. What is requirements.txt?
8. What is pip freeze?
9. What is uv?
10. What is the difference between pip and uv?
11. What is pyproject.toml?
12. What is a lock file?
13. Why should package versions be pinned?
14. How can two projects use different versions of the same package?
15. How do you manage multiple Python versions?

These topics came from your Python deep-dive into venv, pip, uv, typing, Pydantic, iterators, generators, and concurrency. [Past Chat...ith multip | Chat]

---

10. Iterators, Generators, and Context Managers

Subtopics

- Iterable
- Iterator
- iter()
- next()
- Generator functions
- yield
- Generator expressions
- Lazy evaluation
- Context managers
- with statement
- __enter__
- __exit__
- contextlib

Questions

1. What is an iterable?
2. What is an iterator?
3. What is the difference between iterable and iterator?
4. What does iter() do?
5. What does next() do?
6. What is a generator?
7. What is the difference between return and yield?
8. Why are generators memory-efficient?
9. What is lazy evaluation?

10. What is a context manager?
11. Why is the with statement used?
12. How do __enter__ and __exit__ work?
13. How can you create a custom context manager?
14. Where are context managers used in backend applications?

---

11. Sync, Async, Threading, and Multiprocessing

Subtopics

- Synchronous execution
- Asynchronous execution
- Concurrency
- Parallelism
- Event loop
- Coroutines
- async
- await
- asyncio
- Threads
- Processes
- Global Interpreter Lock
- CPU-bound tasks
- I/O-bound tasks

Questions

1. What is synchronous execution?
2. What is asynchronous execution?
3. What is the difference between concurrency and parallelism?
4. What is a coroutine?
5. What do async and await mean?
6. What is an event loop?
7. What is asyncio?
8. What happens while a coroutine is waiting?
9. What is threading?
10. What is multiprocessing?
11. What is the GIL?
12. Why is threading useful for I/O-bound operations?
13. Why is multiprocessing useful for CPU-bound operations?
14. What is the difference between threading, multiprocessing, and asyncio?

15. When should each one be used?
16. Can an asynchronous function contain blocking code?
17. What happens if blocking code is used in an async endpoint?
18. How does FastAPI use asynchronous execution?

You specifically studied the comparison between threading, multiprocessing, and asyncio, including I/O-bound and CPU-bound workloads. [Past Chat...uring APIs | Chat]

Part C: Application Architecture and Design

12. Backend Application Layers

Subtopics

- Presentation/API layer
- Controller or router layer
- Business/service layer
- Data/repository layer
- Configuration layer
- Infrastructure layer
- Domain models
- Database models
- Schemas
- Utilities
- Dependency injection

Questions

1. What is layered architecture?
2. What is the API layer?
3. What is the business layer?
4. What is the data layer?
5. What is the configuration layer?
6. What belongs in the service layer?
7. What belongs in the repository layer?
8. Should SQL queries be written inside API routes?
9. What is separation of concerns?
10. What is dependency injection?
11. Why should business logic be separated from database logic?
12. What is the difference between a Pydantic model and an ORM model?
13. What is the difference between a domain model and database model?

14. How does a request move through different application layers?
15. How does layered architecture improve testing?

---

13. System Boundaries and Component Classification

Subtopics

- Internal components
- External components
- Infrastructure components
- Application services
- Third-party providers
- Databases
- Message brokers
- SMTP providers
- Users and clients
- System ownership

Example classification

Internal application components

- Login API
- Authentication service
- Email service
- Order service

Infrastructure components

- Database
- Kafka
- Cache
- Container platform

External components

- User
- SMTP provider
- Payment gateway
- External identity provider

Questions

1. What is a system boundary?

2. How do you identify internal components?
3. How do you identify external dependencies?
4. Is a database part of application logic or infrastructure?
5. Is Kafka internal or external?
6. Is an SMTP provider internal or external?
7. Who owns each component?
8. What happens when an external provider is unavailable?
9. How should external integrations be abstracted?
10. Why are boundaries important in architecture diagrams?

Your architecture-classification question used a login API, authentication service, database, Kafka, email service, SMTP provider, and user to distinguish application, infrastructure, and external boundaries. [Past Chat...ion inster | Chat]

---

14. Design Patterns and Factory Pattern

Subtopics

- Design patterns
- Factory Method
- Abstract Factory
- Factory mapping
- Strategy pattern
- Dependency inversion
- Loose coupling
- Interfaces
- Abstract classes
- Extensibility
- Open/Closed Principle

Questions

1. What is a design pattern?
2. Why are design patterns used?
3. What is the Factory Method pattern?
4. What problem does Factory Method solve?
5. What is factory mapping?
6. How is a dictionary used for factory mapping?
7. What is loose coupling?
8. What is tight coupling?
9. What is the Open/Closed Principle?
10. How can a system support multiple authentication providers?

11. How would you design Google, Microsoft, and Okta authentication implementations?
12. What is an abstract identity-provider interface?
13. What is the difference between Factory and Strategy patterns?
14. When should the Factory pattern not be used?
15. How does dependency injection relate to Factory patterns?

Factory Method was highlighted in the sessions as a way to keep identity-provider and application code loosely coupled. [Past Chat...tes 8 seco | Chat], [Past Chat...gate the m | Chat], [Past Chat...systems), | Chat]

Part D: Authentication and Security

15. Authentication and Authorization

Subtopics

- Authentication
- Authorization
- Username and password
- API keys
- Basic authentication
- Session-based authentication
- Token-based authentication
- Role-Based Access Control
- Claims
- Permissions
- Multi-Factor Authentication

Questions

1. What is authentication?
2. What is authorization?
3. What is the difference between authentication and authorization?
4. What is an API key?
5. What is Basic Authentication?
6. What is session-based authentication?
7. What is token-based authentication?
8. What is RBAC?
9. What is a role?
10. What is a permission?

11. What is a claim?
12. What is MFA?
13. Where should authentication logic be placed?
14. How should passwords be stored?
15. Why should plain-text passwords never be stored?
16. What is password hashing?
17. What is the difference between hashing and encryption?

---

16. OAuth

Subtopics

- OAuth definition
- Authorization delegation
- Problems before OAuth
- Password sharing
- Limited access
- Token-based permissions
- Resource owner
- Client application
- Authorization server
- Resource server
- Client ID
- Client secret
- Authorization code
- Access token
- Refresh token
- Redirect URI
- Scope
- Consent
- Grant types

OAuth flows

- Authorization Code Flow
- Authorization Code Flow with PKCE
- Client Credentials Flow
- Device Authorization Flow
- Refresh Token Flow

Questions

1. What is OAuth?
2. Why was OAuth created?
3. What problem does OAuth solve?
4. Is OAuth authentication or authorization?
5. What is the Resource Owner?
6. What is the Client Application?
7. What is the Authorization Server?
8. What is the Resource Server?
9. What is a Client ID?
10. What is a Client Secret?
11. Where should a Client Secret be stored?
12. What is an Authorization Code?
13. Why is an Authorization Code short-lived?
14. Why is the Access Token not directly returned in the browser URL?
15. How is the Authorization Code exchanged for an Access Token?
16. Does the client return the same Authorization Code to the provider?
17. What does the provider validate during token exchange?
18. What is a Redirect URI?
19. What is a Scope?
20. What is a Consent Screen?
21. What is an Access Token?
22. What is a Refresh Token?
23. What happens when an Access Token expires?
24. What is PKCE?
25. When is Client Credentials Flow used?
26. What questions should be asked before implementing OAuth?
27. How should tokens be stored?
28. How can OAuth access be revoked?
29. What is the difference between OAuth 1.0 and OAuth 2.0?
30. Why does OAuth 1.0 use request signatures?

OAuth was one of your most detailed learning areas, including Authorization Code exchange, Client ID, Client Secret, Redirect URI, Access Token, Refresh Token, scope, and implementation requirements. [Past Chat...ave an OAu | Chat], [Past Chat...*: Uses ac | Chat], [Past Chat...hould ask | Chat]

---

17. OIDC, SSO, and Identity Providers

Subtopics

- OpenID Connect
- Identity Provider
- Single Sign-On
- ID Token
- Access Token
- User identity
- OAuth and OIDC relationship
- Federated identity
- Enterprise SSO

Questions

1. What is OIDC?
2. Is OAuth 2.0 the same as OIDC?
3. What problem does OIDC solve?
4. What is an ID Token?
5. What is the difference between ID Token and Access Token?
6. How is OIDC built on OAuth 2.0?
7. What is SSO?
8. Where does SSO occur in the OAuth/OIDC flow?
9. Is SSO a protocol?
10. What is an Identity Provider?
11. What is a Service Provider?
12. How does “Continue with Google” work?
13. How does enterprise SSO work?
14. What is federated identity?
15. What is the relationship between OAuth, OIDC, JWT, and SSO?
16. Can OAuth alone be used for login?
17. Why does OIDC include the openid scope?

---

18. JWT and Digital Signatures

Subtopics

- JSON Web Token
- Header
- Payload
- Signature

- Claims
- Registered claims
- Token expiry
- Token verification
- Symmetric and asymmetric signatures
- Bearer tokens
- Token revocation
- Access Token and ID Token

Questions

1. What is JWT?
2. What are the three parts of a JWT?
3. What is stored in the header?
4. What is stored in the payload?
5. What are claims?
6. What is the JWT signature?
7. How does the signature prevent tampering?
8. Is JWT encrypted?
9. Can anyone decode a JWT?
10. What is the difference between decoding and verifying?
11. What does the exp claim mean?
12. What does the sub claim mean?
13. What does the aud claim mean?
14. What does the iss claim mean?
15. What is the difference between symmetric and asymmetric signing?
16. What is a Bearer token?
17. Can an Access Token be a JWT?
18. Is an OIDC ID Token normally a JWT?
19. How should JWT expiry be handled?
20. How can JWTs be revoked before expiry?
21. What is token rotation?

Your questions connected OAuth 1.0 signatures, OAuth 2.0 Bearer tokens, OIDC ID Tokens, JWT structure, and signature verification. [Past Chat...*: Uses ac | Chat]

Part E: Databases and SQL

19. Database Fundamentals and Persistence

Subtopics

- Database
- DBMS
- RDBMS
- Persistence
- Volatile and non-volatile storage
- SQL databases
- NoSQL databases
- In-memory databases
- Data warehouses
- Time-series databases
- Graph databases

Questions

1. What is a database?
2. What is DBMS?
3. What is RDBMS?
4. Why must databases be persistent?
5. What is the difference between RAM and disk storage?
6. What happens to in-memory data after an application restart?
7. What is the difference between SQL and NoSQL?
8. When should a relational database be used?
9. When should a document database be used?
10. What is a key-value database?
11. What is an in-memory database?
12. What is a graph database?
13. What is a time-series database?
14. What is a data warehouse?
15. How do databases recover after failures?

---

20. Data Modelling

Subtopics

- Entity
- Attribute

- Primary key
- Foreign key
- Unique key
- Composite key
- Relationships
- Constraints
- Conceptual model
- Logical model
- Physical model
- Entity Relationship Diagram

Relationships

- One-to-one
- One-to-many
- Many-to-many

Order-management entities

- Customer
- Product
- Order
- OrderItem
- Payment

Questions

1. What is data modelling?
2. Why is data modelling required?
3. What is an entity?
4. What is an attribute?
5. What is a primary key?
6. What is a foreign key?
7. What is a composite key?
8. What is a unique constraint?
9. What is a one-to-one relationship?
10. What is a one-to-many relationship?
11. What is a many-to-many relationship?
12. How is a many-to-many relationship implemented?
13. Why is OrderItem required?
14. What is the relationship between Customer and Order?
15. What is the relationship between Order and OrderItem?
16. What is the relationship between Product and OrderItem?

17. Is Order-to-Payment one-to-one or one-to-many?
18. What is the difference between conceptual, logical, and physical models?
19. What is an ER diagram?
20. How should delete behavior be handled for related entities?

Your order-system model covered Customer, Product, Order, OrderItem, and Payment, including one-to-many and many-to-many relationships. [Past Chat...systems), | Chat]

---

21. Normalization

Subtopics

- Data redundancy
- Insert anomaly
- Update anomaly
- Delete anomaly
- First Normal Form
- Second Normal Form
- Third Normal Form
- Boyce-Codd Normal Form
- Denormalization

Questions

1. What is normalization?
2. Why is normalization used?
3. What is data redundancy?
4. What is an insert anomaly?
5. What is an update anomaly?
6. What is a delete anomaly?
7. What is 1NF?
8. What is 2NF?
9. What is 3NF?
10. What is a partial dependency?
11. What is a transitive dependency?
12. What is BCNF?
13. What is denormalization?
14. When should data be denormalized?
15. How would you normalize an order-management database?

---

22. SQL Fundamentals to Advanced

Subtopics

SQL command categories

- DDL: CREATE, ALTER, DROP, TRUNCATE
- DML: INSERT, UPDATE, DELETE
- DQL: SELECT
- TCL: COMMIT, ROLLBACK, SAVEPOINT
- DCL: GRANT, REVOKE

Querying

- WHERE
- AND, OR, NOT
- IN
- BETWEEN
- LIKE
- ORDER BY
- DISTINCT
- LIMIT
- CASE

Aggregation

- COUNT
- SUM
- AVG
- MIN
- MAX
- GROUP BY
- HAVING

Advanced SQL

- Subqueries
- Correlated subqueries
- Common table expressions
- Recursive CTEs
- Window functions
- Stored procedures
- Views
- Triggers
- Query plans

- Locks
- Isolation levels

Questions

1. What is SQL?
2. What are DDL, DML, DQL, TCL, and DCL?
3. What are CRUD operations?
4. What is the difference between DELETE, DROP, and TRUNCATE?
5. What is the difference between WHERE and HAVING?
6. What is GROUP BY?
7. What is a subquery?
8. What is a correlated subquery?
9. What is a CTE?
10. What is a recursive CTE?
11. What are window functions?
12. What is the difference between ROW_NUMBER, RANK, and DENSE_RANK?
13. What is a view?
14. What is a stored procedure?
15. What is a trigger?
16. How do you find duplicate records?
17. How do you find the second-highest salary?
18. How do you find customers who have not placed orders?
19. How do you calculate running totals?
20. How do you investigate a slow SQL query?

---

23. SQL Joins

Subtopics

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL OUTER JOIN
- CROSS JOIN
- SELF JOIN
- Multiple joins
- Join conditions
- Join performance

Questions

1. What is a SQL join?
2. What does INNER JOIN return?
3. What does LEFT JOIN return?
4. What does RIGHT JOIN return?
5. What does FULL OUTER JOIN return?
6. Does MySQL directly support FULL OUTER JOIN?
7. How can FULL OUTER JOIN be simulated in MySQL?
8. What is a CROSS JOIN?
9. What is a SELF JOIN?
10. How do you find customers without orders?
11. What happens if the join condition is missing?
12. How do indexes improve joins?
13. What is the difference between a join and a subquery?
14. How do duplicate rows arise during joins?

---

24. Indexes

Subtopics

- Full table scan
- B-tree index
- Primary index
- Unique index
- Single-column index
- Composite index
- Full-text index
- Index selectivity
- Leftmost-prefix rule
- Covering index
- Query execution plan
- Index overhead

Questions

1. What is an index?
2. Why is an index used?
3. What is a full-table scan?
4. What is a B-tree index?
5. What is a unique index?
6. What is a composite index?

7. What is the leftmost-prefix rule?
8. What is a covering index?
9. What is index selectivity?
10. How do indexes improve filtering?
11. How do indexes improve joins?
12. Why can too many indexes be harmful?
13. Why do indexes slow INSERT and UPDATE operations?
14. When will a database ignore an index?
15. How do you inspect a query execution plan?
16. Which columns normally need indexes?

---

25. Transactions and ACID

Subtopics

- Transaction
- BEGIN
- COMMIT
- ROLLBACK
- SAVEPOINT
- Atomicity
- Consistency
- Isolation
- Durability
- Concurrency
- Dirty reads
- Non-repeatable reads
- Phantom reads
- Isolation levels
- Deadlocks

Questions

1. What is a database transaction?
2. Why are transactions required?
3. What is COMMIT?
4. What is ROLLBACK?
5. What is SAVEPOINT?
6. What does ACID stand for?
7. What is Atomicity?
8. What is Consistency?

9. What is Isolation?
10. What is Durability?
11. What is a dirty read?
12. What is a non-repeatable read?
13. What is a phantom read?
14. What are the common transaction-isolation levels?
15. What is a database lock?
16. What is a deadlock?
17. How can deadlocks be reduced?
18. How would you implement a bank transfer transaction?
19. Should an external API call be made inside a database transaction?
20. What happens if the application crashes before COMMIT?

Database persistence, types, data modelling, order relationships, normalization, joins, indexes, transactions, ACID, and the SQL learning roadmap formed one large continuous learning thread. [Past Chat...systems), | Chat]

---

26. ORM, Database Connections, and Migrations

Subtopics

- Object Relational Mapping
- Models and tables
- ORM queries
- Raw SQL
- Database engine
- Database sessions
- Connections
- Connection pool
- Transactions
- Lazy loading
- Eager loading
- N+1 query problem
- Schema migrations
- Alembic
- SQLAlchemy

Questions

1. What is ORM?
2. Why is ORM used?

3. How does ORM convert objects into SQL?
4. What is the difference between ORM and raw SQL?
5. What is a database engine?
6. What is a database session?
7. What is a database connection?
8. What is connection pooling?
9. Why is creating a new connection for every request inefficient?
10. What is pool size?
11. What happens when the connection pool is exhausted?
12. What is lazy loading?
13. What is eager loading?
14. What is the N+1 query problem?
15. What is a database migration?
16. Why are migrations required?
17. What is Alembic?
18. How should database sessions be handled in FastAPI?
19. When should raw SQL be preferred over ORM?
20. How should ORM transactions be managed?

Part F: Docker and Deployment

27. Docker

Subtopics

- Docker
- Containers
- Images
- Dockerfile
- Build context
- Layers
- Base images
- Ports
- Volumes
- Environment variables
- Networks
- Docker Compose
- Container logs
- Container lifecycle

Questions

1. What is Docker?
2. Why is Docker used?
3. What problem does Docker solve?
4. What is a Docker image?
5. What is a Docker container?
6. What is the difference between an image and a container?
7. What is a Dockerfile?
8. What does FROM do?
9. What does COPY do?
10. What does RUN do?
11. What is the difference between CMD and ENTRYPOINT?
12. What is port mapping?
13. What is a Docker volume?
14. Why are volumes required for persistent data?
15. What is a Docker network?
16. What is Docker Compose?
17. How do application and database containers communicate?
18. How do you inspect container logs?
19. How do you pass secrets into containers?
20. Why should a database not store its data only inside the container filesystem?

Docker was included in the completed syllabus and later connected to running local Kafka and backend services. [Past Chat...ashraj Pat | Chat], [Past Chat...1 Bhavesh | Chat]

---

28. Cloud Deployment and Cloud Run

Subtopics

- Container deployment
- Managed services
- Stateless applications
- Environment variables
- Secrets
- Service URL
- Scaling
- Health checks
- Logging
- Database connectivity
- Continuous deployment

- Cold starts

Questions

1. What is a managed cloud service?
2. What is Cloud Run?
3. How is a FastAPI application deployed as a container?
4. Why should cloud applications be stateless?
5. Where should application configuration be stored?
6. How should secrets be managed?
7. How does automatic scaling work?
8. What is a cold start?
9. How does a deployed application connect to a database?
10. How are logs collected in the cloud?
11. How can deployment be automated from source control?
12. What should a production Docker image contain?
13. What is the difference between deploying a virtual machine and a container?
14. What happens when multiple application instances start?

Part G: System Design

29. Synchronous and Asynchronous Processing

Subtopics

- Synchronous request
- Asynchronous request
- Blocking operations
- Non-blocking operations
- Background jobs
- Queues
- Event-driven processing
- Delayed processing
- Retry handling

Questions

1. What is synchronous processing?
2. What is asynchronous processing?
3. When should a request be synchronous?
4. When should a task be asynchronous?
5. Should login be synchronous or asynchronous?

6. Should sending a welcome email be synchronous or asynchronous?
7. What happens if an email service is slow?
8. How does a queue help asynchronous processing?
9. What response should an API return for accepted background work?
10. How does asynchronous processing improve user experience?
11. What are the disadvantages of asynchronous processing?
12. How is failure communicated back to the user?

---

30. Database vs Queue

Subtopics

- Database persistence
- Queue buffering
- Event storage
- Immediate updates
- Delayed processing
- Producer-consumer communication
- Retry handling
- Failure recovery

Questions

1. What is the difference between a database and a queue?
2. When should data be written directly to a database?
3. When should an event be placed in a queue?
4. Can a queue replace a database?
5. Can a database act as a queue?
6. Why should email requests be placed in a queue?
7. What happens if the consumer is unavailable?
8. How does a queue prevent data loss?
9. What is an outbox pattern?
10. How do you ensure that both database write and event publication succeed?

---

31. Email Notification Architecture

Subtopics

- API
- Business service
- Database

- Kafka or message queue
- Email consumer
- SMTP provider
- Retry mechanism
- Dead-letter queue
- Email templates
- Delivery status

Questions

1. How should an email-notification system be designed?
2. Why should the main API not wait for the email to be sent?
3. What component acts as the producer?
4. What component acts as the consumer?
5. What information should be placed in the event?
6. What happens if the email service is down?
7. What happens if the SMTP provider is unavailable?
8. How should email retries work?
9. How do you prevent duplicate emails?
10. What is a dead-letter queue?
11. How should delivery status be stored?
12. Where should email templates be maintained?

---

32. Rate Limiting

Subtopics

- Request limits
- User-based limits
- IP-based limits
- API-key-based limits
- Fixed window
- Sliding window
- Token bucket
- Leaky bucket
- HTTP 429
- Distributed rate limiting
- Redis
- DDoS and API protection

Questions

1. What is rate limiting?
2. Why is rate limiting required?
3. What happens when 1,000 authentication requests arrive together?
4. What is a fixed-window rate limiter?
5. What is a sliding-window rate limiter?
6. What is the token-bucket algorithm?
7. What is the leaky-bucket algorithm?
8. What status code should be returned after exceeding a limit?
9. What is the Retry-After header?
10. Should limits be based on IP address or user identity?
11. How is distributed rate limiting implemented?
12. Why is Redis commonly used for rate limiting?
13. What is the difference between rate limiting, throttling, and concurrency limiting?

The system-design sessions specifically connected authentication traffic, rate limiting, sync/async decisions, queue usage, and email-notification delivery. [Past Chat...al-text="t | Chat], [Past Chat...1 Bhavesh | Chat]

Part H: Kafka

33. Kafka Fundamentals

Subtopics

- Event streaming
- Message broker
- Producer
- Consumer
- Topic
- Partition
- Offset
- Consumer group
- Broker
- Cluster
- Replication factor
- Leader and follower
- Retention
- Ordering

- Delivery semantics
- Rebalancing

Questions

1. What is Kafka?
2. Why is Kafka used?
3. What problem does Kafka solve?
4. Is Kafka a queue or an event-streaming platform?
5. What is a Kafka broker?
6. What is a Kafka cluster?
7. What is a topic?
8. What is a partition?
9. Why are topics divided into partitions?
10. What is an offset?
11. What is a producer?
12. What is a consumer?
13. What is a consumer group?
14. What is a replication factor?
15. What are leaders and followers?
16. What is message retention?
17. Does Kafka delete a message immediately after consumption?
18. Does Kafka follow FIFO?
19. Is ordering guaranteed across the entire topic?
20. What happens if a consumer goes down?
21. What happens when the consumer restarts?
22. What is consumer rebalancing?
23. What happens when a broker goes down?
24. What are at-most-once, at-least-once, and exactly-once delivery?
25. What is an acknowledgement?
26. What is an idempotent producer?
27. What are duplicate events?
28. What is a dead-letter topic?
29. What is the difference between Kafka and RabbitMQ?

---

34. Kafka Producer and Consumer in Real Applications

Real components

Possible producers

- Order service

- Registration service
- Payment service
- Transaction service
- Driver-location service

Possible consumers

- Email service
- Inventory service
- Analytics service
- Audit service
- Fraud-detection service
- Reporting service

Questions

1. Is the producer part of Kafka?
2. Is the consumer part of Kafka?
3. What is the producer in an order-management system?
4. What are the consumers of an OrderCreated event?
5. Can one producer publish to multiple topics?
6. Can multiple producers publish to the same topic?
7. Can multiple consumers read the same event?
8. What happens when consumers are in different consumer groups?
9. What happens when consumers are in the same consumer group?
10. How do multiple consumers share work?
11. Can two consumers in the same group read the same partition simultaneously?
12. What happens when consumers are more than partitions?
13. What happens when partitions are more than consumers?
14. How does Kafka assign partitions to consumers?
15. How do multiple service instances scale consumption?

---

35. Kafka Partitioning, Replication, and Ordering

Subtopics

- Partition key
- Message distribution
- Partition-level ordering
- Parallel processing
- Replication
- Fault tolerance

- Partition leader
- Replica followers
- Consumer mapping

Questions

1. What is partitioning?
2. Why does partitioning improve throughput?
3. How does partitioning allow parallel processing?
4. How does the producer select a partition?
5. What happens if no message key is provided?
6. What happens if a key is provided?
7. How can all events for one order remain ordered?
8. Is partitioning the same as replication?
9. What is the purpose of replication?
10. How does replication provide fault tolerance?
11. What happens if the partition leader fails?
12. Can the same message exist in multiple replicas?
13. Does each consumer receive a replica copy?
14. How many active consumers can process a topic with three partitions?
15. What happens when a partition count is increased?

---

36. Kafka Practical Setup

Subtopics

- Kafka local installation
- Docker-based Kafka
- Kafka broker configuration
- Topic creation
- Partition configuration
- Producer commands
- Consumer commands
- Consumer groups
- Offsets
- Docker Compose
- Python Kafka client
- FastAPI and Kafka integration

Questions

1. How do you run Kafka locally?

2. How do you run Kafka using Docker?
3. How do you create a topic?
4. How do you specify the number of partitions?
5. How do you configure replication factor?
6. How do you list Kafka topics?
7. How do you publish test messages?
8. How do you consume test messages?
9. How do you create a consumer group?
10. How do you inspect consumer-group offsets?
11. How do you connect a Python application to Kafka?
12. How do you publish an event from FastAPI?
13. How should a Kafka consumer handle failures?
14. How should offset commits be managed?
15. How do you prevent the application from blocking while publishing?
16. How should message schemas be versioned?

The latest sessions concentrated on Kafka recap, local setup, topics, partitions, producers, consumers, consumer groups, parallel processing, Docker, and partition-versus-replication clarification. [Past Chat...1 Bhavesh | Chat], [Past Chat...al-text="t | Chat]

Part I: Performance, Troubleshooting, and Observability

37. Backend Troubleshooting

Subtopics

- Reproducing an issue
- Understanding the user problem
- Release comparison
- Logs and error messages
- Database investigation
- External-service failures
- Configuration issues
- Network issues
- Root-cause analysis
- Rollback and mitigation

Questions

1. What should you do when a user reports an application issue?
2. Why should the issue be reproduced first?

3. How do you verify whether the reported behavior is actually a defect?
4. How do you identify whether a recent release caused the issue?
5. What should be checked in application logs?
6. How do you trace one request across multiple services?
7. How do you identify whether the database is causing slowness?
8. How do you inspect external API failures?
9. How do you distinguish code, configuration, database, and infrastructure problems?
10. When should a release be rolled back?
11. What information should be collected before changing production code?
12. How do you perform root-cause analysis?

---

38. Performance Measurement

Subtopics

- Latency
- Throughput
- Response time
- Execution time
- Percentiles
- Bottlenecks
- Profiling
- Database query time
- External API time
- Network latency
- Application latency
- Load testing
- Concurrency testing

Questions

1. What is latency?
2. What is throughput?
3. What is the difference between latency and response time?
4. How do you measure code execution time?
5. How do you measure database-query time?
6. How do you measure external API time?
7. What are P50, P95, and P99 latency?
8. Why is average latency sometimes misleading?
9. How do you identify the slowest application layer?
10. What happens when application and database are in different environments?

11. Why is the same-environment connection usually faster?
12. Can application and database operations run concurrently?
13. What is connection-pool latency?
14. What is load testing?
15. What is stress testing?
16. What is profiling?
17. How do you investigate an endpoint that became slow after a release?

---

39. Logging and Observability

Subtopics

- Logs
- Metrics
- Traces
- Structured logging
- Log levels
- Correlation IDs
- Request IDs
- Trace IDs
- Request duration
- Database duration
- Error rate
- Service health
- Dashboards
- Alerts
- Sensitive data in logs

Questions

1. What is observability?
2. What are logs?
3. What are metrics?
4. What are distributed traces?
5. What is the difference between logs, metrics, and traces?
6. What are INFO, DEBUG, WARNING, and ERROR levels?
7. What should be logged for an incoming request?
8. What should not be logged?
9. Why should PII not be logged?
10. What is a correlation ID?
11. What is a request ID?

12. What is a trace ID?
13. How do you trace a request across microservices?
14. How do you measure total request duration?
15. How do you isolate database duration from total duration?
16. How do metrics help identify performance problems?
17. What alerts should be configured?
18. Can logs, traces, and metrics be committed to GitHub?
19. Why is Git not an appropriate log-storage system?
20. Where should production telemetry be stored?

The troubleshooting discussions focused on reproducing issues, comparing recent releases, measuring request and database durations, and using logs, traces, and metrics to isolate a slow layer. [Past Chat...gate the m | Chat], [Past Chat...uring APIs | Chat]

Part J: Additional Subjects You Studied

40. Excel VBA Automation

Subtopics

- VBA definition
- Macros
- Sub procedures
- Functions
- Variables
- Loops
- Conditions
- Excel object model
- Workbook and worksheet objects
- Cells and ranges
- Buttons
- Form controls
- UserForms
- TextBoxes
- CommandButtons
- Saving records in worksheets
- VBA error debugging

Questions

1. What is VBA?

2. Why is VBA used in Excel?
3. What is a macro?
4. What is the difference between Sub and Function?
5. How do you create a macro?
6. How do you run a macro?
7. How do you classify numbers as odd or even?
8. How do you loop through worksheet rows?
9. How do you find the last used row?
10. How do you add a button manually?
11. How do you create a button using VBA?
12. How do you assign a macro to a button?
13. What causes “Sub or Function not defined”?
14. Why must procedures be placed in a standard module?
15. Why should variables such as i and ws be declared?
16. How do you create a UserForm?
17. How do you store UserForm values in the next empty row?
18. How do you clear fields after saving?
19. Why should a workbook be saved as .xlsm?
20. What is FP&A with VBA?

Your VBA thread covered odd/even automation, one-click buttons, creating buttons programmatically, debugging procedure errors, and building a UserForm that stores entries in a worksheet. [Past Chat..."Even"** i | Chat], [Past Chat...k**. ### W | Chat]

---

41. Artificial Intelligence and Machine Learning Foundations

Subtopics

- Artificial Intelligence
- Machine Learning
- Deep Learning
- Neural networks
- Reinforcement learning
- Probabilistic models
- Regular software versus AI systems
- AI history
- Big Data and AI
- Ethical AI
- Reliable and unreliable AI conditions
- AI libraries

- Data preparation
- Model training
- Model evaluation
- Explainability
- Bias

Questions

1. What is AI?
2. What is Machine Learning?
3. What is Deep Learning?
4. What is the difference between AI, ML, and Deep Learning?
5. How is regular software different from an AI system?
6. What is a neural network?
7. What is reinforcement learning?
8. What is a probabilistic model?
9. How does Big Data benefit AI?
10. What are common AI-development risks?
11. What is AI bias?
12. What is ethical AI?
13. What is federated learning?
14. Why is human oversight important?
15. What makes an AI use case reliable or unreliable?
16. What is explainability?
17. What AI tools and libraries are commonly used?
18. What are the common steps in an ML workflow?
19. Why must data be analysed before model training?
20. How should missing values be handled?

---

42. Python AI Development Tools

Subtopics

- Python for AI
- Jupyter Notebook
- JupyterLab
- Anaconda
- Google Colab
- NumPy
- Pandas
- Matplotlib

- Plotly
- Scikit-learn
- TensorFlow
- PyTorch
- OpenCV
- CUDA
- Data visualization
- Model building and training

Questions

1. Why is Python popular for AI?
2. What is Jupyter Notebook?
3. What is JupyterLab?
4. What is Anaconda?
5. What are the benefits of Anaconda?
6. What is Google Colab?
7. What is NumPy?
8. What is Pandas?
9. What is Matplotlib?
10. What is Plotly Express?
11. How do you create a scatter plot?
12. What is Scikit-learn?
13. What is TensorFlow?
14. What is PyTorch?
15. What is OpenCV?
16. What is CUDA?
17. What command trains a model?
18. What is the difference between compiling, fitting, and predicting?
19. What is a confusion matrix?
20. What is feature importance?
21. What is hyperparameter tuning?

Your course questions covered Python AI development environments, missing values, visualization, ML algorithms, CNNs, Gradient Boosting, model training, evaluation, and computer-vision tools. [Past Chat...i><li valu | Chat]

---

43. Human-Computer Interaction

Subtopics

- Purpose of HCI

- HCI objectives
- Usability
- User satisfaction
- Intuitive design
- User studies
- Interface studies
- Interaction studies
- Computer studies
- User types
- User personas
- Task analysis
- Mock-ups
- Prototypes
- Usability testing
- Design tools
- Evaluation tools
- Implementation tools
- HCI disciplines
- History of HCI
- Recent HCI trends

Questions

1. What is HCI?
2. What are the main objectives of HCI?
3. What is usability?
4. What makes an interface intuitive?
5. What is the purpose of user studies?
6. What is an interface study?
7. What is an interaction study?
8. What is a computer study?
9. How are user types defined?
10. What is a user persona?
11. Why is user feedback important?
12. What is task analysis?
13. Why are mock-ups used?
14. How do mock-ups help developers?
15. What is prototyping?
16. What is usability testing?
17. What are HCI design tools?
18. What are HCI evaluation tools?

19. What are HCI implementation tools?
20. Which disciplines contribute to HCI?
21. How do psychology, ergonomics, computer science, and engineering contribute?
22. How has HCI evolved historically?
23. What are the recent trends in HCI?
24. How are mobile interfaces, VR/AR, speech interaction, and brain-computer interfaces related to HCI?
25. What are the success factors of a user-oriented approach?

Your HCI assessments repeatedly covered objectives, disciplines, user types, mock-ups, tools, historical stages, user-oriented design, task definitions, and recent interface trends. [Past Chat...ls** ### E | Chat], [Past Chat...re high an | Chat], [Past Chat...ck-end** # | Chat], [Past Chat...lue="4">11 | Chat]

---

44. Prompt Engineering and LLMs

Subtopics

- Large Language Models
- Prompts
- Zero-shot prompting
- One-shot prompting
- Few-shot prompting
- Role-based prompting
- Contextual prompting
- Instruction prompting
- Step-by-step prompting
- Self-consistency
- ReAct
- Prompt constraints
- Output formatting

Questions

1. What is an LLM?
2. What is prompt engineering?
3. What is zero-shot prompting?
4. What is one-shot prompting?
5. What is few-shot prompting?
6. What is role-based prompting?
7. What is contextual prompting?
8. What is instruction prompting?

9. What is step-by-step prompting?
10. What is self-consistency prompting?
11. What is ReAct prompting?
12. How do examples improve results?
13. Why should constraints be included in prompts?
14. How do you request a specific output format?
15. How can prompt engineering be used for backend debugging?
16. How can an LLM assist in analysing logs?

The prompt-engineering discussion covered zero-shot, one-shot, few-shot, role-based, contextual, instruction, step-by-step, self-consistency, and ReAct prompting. [Past Chat...ero-Shot P | Chat]

Final Consolidated Learning Order

For training and interview preparation, revise these subjects in this order:

Level 1: Foundations

1. Python fundamentals
2. OOP
3. Client-server architecture
4. HTTP request-response lifecycle
5. REST APIs
6. Git and environment basics
7. Virtual environments and package management

Level 2: Backend Development

1. FastAPI
2. Pydantic validation
3. Middleware and error handling
4. Layered architecture
5. SQL fundamentals
6. Data modelling
7. ORM and database connections
8. Transactions and ACID

Level 3: Security and Integration

1. Authentication and authorization
2. OAuth 2.0
3. OIDC

4. SSO
5. JWT
6. External-service integration
7. Secrets and token security

Level 4: Production and System Design

1. Docker
2. Cloud deployment
3. Sync versus async
4. Database versus queue
5. Email-notification architecture
6. Rate limiting
7. Kafka
8. Performance measurement
9. Logging, metrics, and tracing
10. Production troubleshooting

Level 5: Advanced Preparation

1. Design patterns
2. Scalability
3. Consumer groups and Kafka partitioning
4. Database optimization
5. Distributed-system failures
6. Reliability and observability

Most Important Questions for Your Next Backend Session

If you have limited preparation time, prioritize these:

1. Explain the complete HTTP request lifecycle.
2. Explain FastAPI architecture from route to database.
3. What is the difference between authentication and authorization?
4. Explain OAuth 2.0 Authorization Code Flow.
5. Explain OAuth, OIDC, SSO, and JWT separately.
6. What is the purpose of the Client Secret?
7. What is database persistence?
8. Explain one-to-one, one-to-many, and many-to-many relationships.
9. Explain normalization, joins, indexes, transactions, and ACID.

10. What is ORM and connection pooling?
11. Explain synchronous versus asynchronous processing.
12. When would you use a database and when would you use a queue?
13. Design an asynchronous email-notification system.
14. What is Kafka and why is it used?
15. What are real producers and consumers?
16. What is a topic, partition, offset, and consumer group?
17. What happens with one producer and multiple consumers?
18. What happens when consumers are in the same versus different groups?
19. What is the difference between partitioning and replication?
20. Does Kafka guarantee FIFO ordering?
21. What happens when a consumer or broker goes down?
22. What is rate limiting?
23. How would you debug a slow endpoint?
24. How do logs, metrics, and traces differ?
25. How do you identify whether the database is the slow layer?

How this master list was improved

- Related questions were grouped under one topic rather than repeated across multiple sessions.
- Your direct follow-up questions were retained as preparation questions.
- Training topics were separated from course-assessment and VBA topics.
- The order now follows an actual backend learning path: **Python → HTTP → FastAPI → Database → Authentication → Architecture → Kafka → Production troubleshooting**.